

# Systeme Intelligent Multi-Modèles pour la Maintenance Prédictive Industrielle

Projet M2 Data Science — EFREI 2025-26

*RNCP36739 — Bloc 4 : Implémenter des méthodes d'IA*

## Problème → Solution

---

| ✗ Maintenance corrective     | ✓ Maintenance prédictive                     |
|------------------------------|----------------------------------------------|
| Panne → réparation d'urgence | Détection 24h avant → intervention planifiée |
| Arrêt non planifié coûteux   | Intervention préventive moins chère          |
| Risque de sécurité           | Zéro surprise                                |

### Notre réponse

Prédire `failure_within_24h` à partir des données capteurs

→ Classification binaire supervisée : panne dans les 24h ? **OUI / NON**

### Dataset

24 042 observations | 4 types de machines (CNC, Pump, Compressor, Robotic Arm)

9 features capteurs : vibration, température, RPM, pression, courant, mode...

# EDA — Ce que les données nous ont appris

## Déséquilibre des classes

Pas de panne 85.2% – 20 482 obs.  
Panne < 24h 14.8% – 3 560 obs.  
Ratio = 5.75 : 1

⚠ Un modèle qui dit "jamais de panne" aurait **85% d'Accuracy** mais détecterait **0 panne**  
→ L'Accuracy est une métrique trompeuse ici

## Métriques choisies

| Métrique | Formule                           | Pourquoi                                                        |
|----------|-----------------------------------|-----------------------------------------------------------------|
| Recall   | $TP / (TP + FN)$                  | Minimiser les pannes ratées (FN = panne manquée = très coûteux) |
| F1-Score | $2 \times (P \times R) / (P + R)$ | Compromis Precision / Recall                                    |
| ROC-AUC  | Aire sous courbe ROC              | Performance globale, indépendante du seuil                      |

## Pipeline & Anti-Leakage

---

Dataset brut (24 042 lignes, 15 colonnes)

▼ Suppression colonnes leakage : failure\_type · rul\_hours · repair\_cost  
| → Ces colonnes "révèlent" la réponse → résultats artificiellement bons

▼ Split STRATIFIÉ 80/20

|       |             |
|-------|-------------|
| TRAIN | 19 233 obs. |
|-------|-------------|

|      |            |
|------|------------|
| TEST | 4 809 obs. |
|------|------------|

▼ ColumnTransformer (ajusté sur TRAIN → appliqué sur TEST)  
├ 7 variables numériques : Median Imputer + StandardScaler  
└ 2 variables catégorielles : Mode Imputer + OneHotEncoder

**Règle d'or :** toutes les statistiques de preprocessing sont calculées sur le train set uniquement.

Tester sur des données qui ont influencé l'entraînement = tricher.

## Les 4 modèles

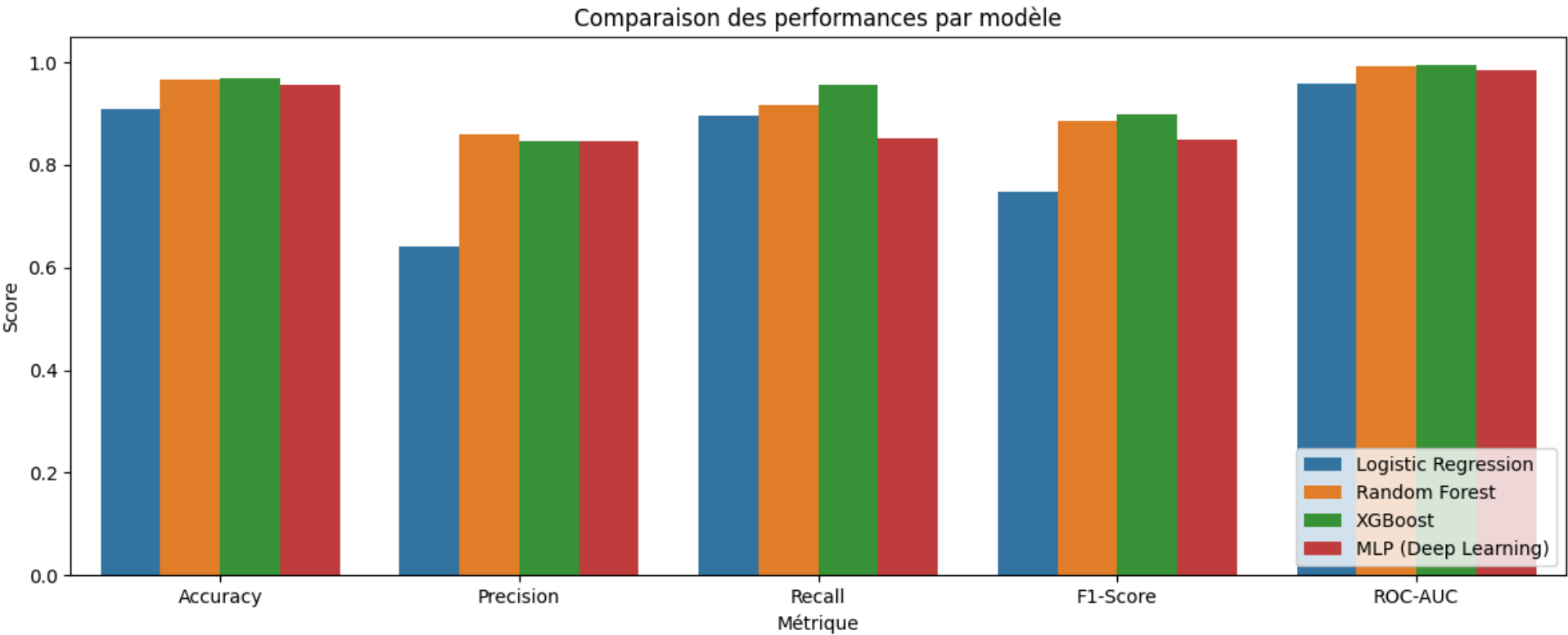
| # | Modèle              | Principe                                                                       | Gestion déséquilibre                 |
|---|---------------------|--------------------------------------------------------------------------------|--------------------------------------|
| 1 | Logistic Regression | Score linéaire → sigmoïde → probabilité                                        | <code>class_weight='balanced'</code> |
| 2 | Random Forest       | 200 arbres indépendants → vote ( <b>bagging</b> )                              | <code>class_weight='balanced'</code> |
| 3 | XGBoost             | 200 arbres séquentiels → chaque arbre corrige le précédent ( <b>boosting</b> ) | <code>scale_pos_weight=5.75</code>   |
| 4 | MLP Deep Learning   | 128 → 64 → 32 neurones, ReLU, backpropagation                                  | <code>early_stopping=True</code>     |

## Bagging vs Boosting

```
BAGGING : arbre1 ┌  
(Random Forest) │ arbre2 ─┐ → vote (parallèle, indépendant)  
                │ arbre3 ─┘
```

```
BOOSTING : arbre1 → corrige → arbre2 → corrige → arbre3 → ... (séquentiel)  
(XGBoost)
```

# Résultats comparatifs



| Modèle                                                                                      | Recall | F1-Score | ROC-AUC |
|---------------------------------------------------------------------------------------------|--------|----------|---------|
| Logistic Regression                                                                         | 0.895  | 0.747    | 0.959   |
| Random Forest                                                                               | 0.916  | 0.887    | 0.993   |
| XGBoost  | 0.955  | 0.898    | 0.996   |
| MLP Deep Learning                                                                           | 0.853  | 0.850    | 0.984   |

# Courbes ROC & Validation

## Courbes ROC — 4 modèles

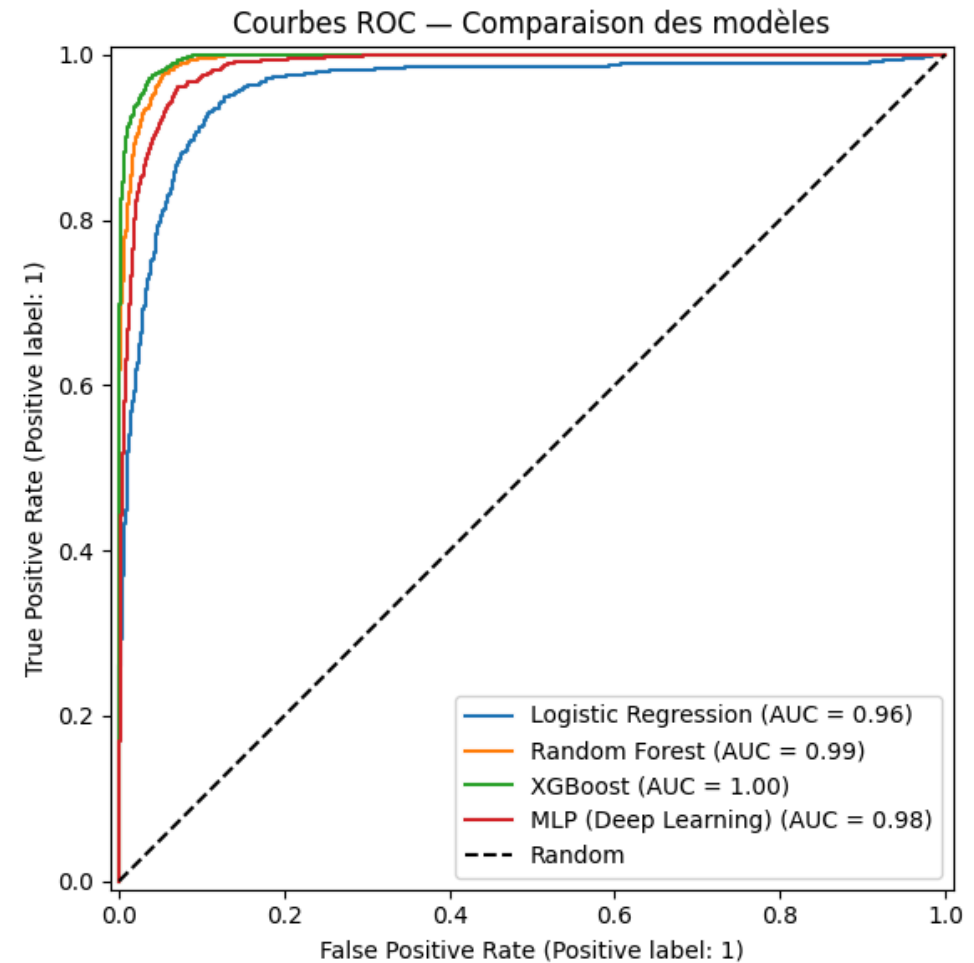
- Plus la courbe est proche du **coin supérieur gauche** = meilleur
- La diagonale = modèle aléatoire
- XGBoost : **AUC = 0.996**

## Cross-validation 5-fold (XGBoost)

```
Fold 1 : 0.893  
Fold 2 : 0.892  
Fold 3 : 0.911  
Fold 4 : 0.901  
Fold 5 : 0.917
```

Moy :  $0.9026 \pm 0.0099$

→ Écart-type faible = modèle **stable**, pas d'overfitting



# Matrice de confusion — XGBoost

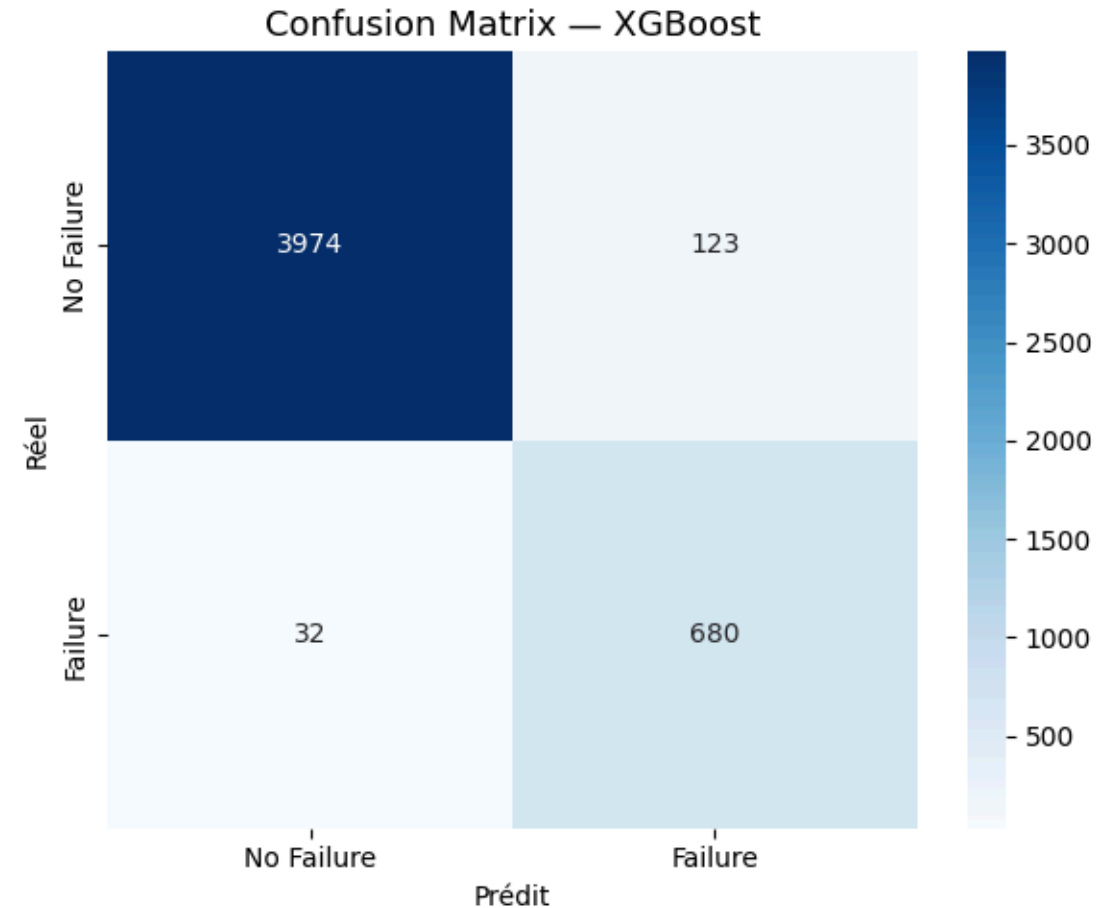
## Lecture

|                     | Prédit : pas de panne | Prédit : panne       |
|---------------------|-----------------------|----------------------|
| Réel : pas de panne | ✓ TN                  | ✗ FP (fausse alerte) |
| Réel : panne        | ✗ FN (panne ratée !)  | ✓ TP                 |

## Résultat XGBoost

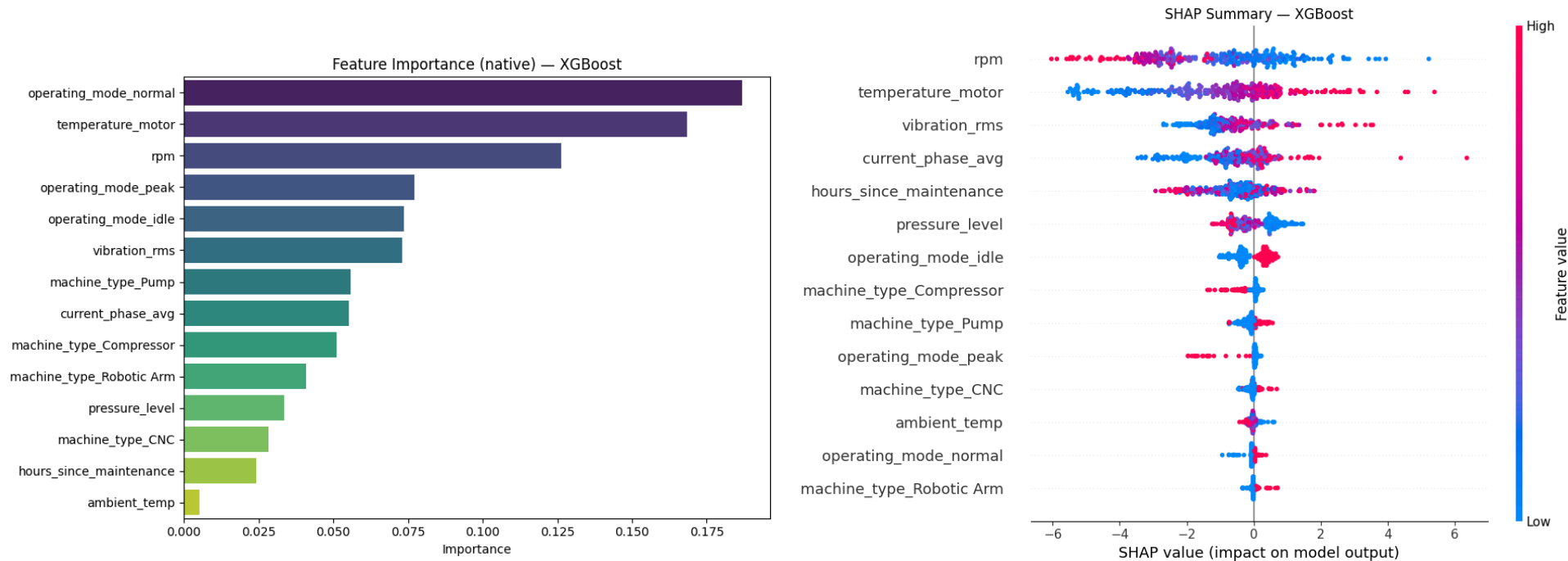
- ~3 399 pannes détectées (TP)
- ~161 pannes manquées (FN)
- → **Recall = 95.5%** = 19 pannes sur 20 détectées

Le FN est le cas le plus coûteux industriellement → on minimise





# Interprétabilité — Feature Importance & SHAP



Top 5 features : vibration\_rms · temperature\_motor · hours\_since\_maintenance · rpm · pressure\_level

SHAP : chaque point = 1 observation | ● valeur élevée → augmente le risque | Position droite → panne probable  
"Pourquoi cette machine est à risque ?" → vibration anormalement élevée → vérifier les roulements

# Dashboard Streamlit & Conclusion

## 4 onglets décisionnels

| Onglet                                                                                             | Contenu                                                                           |
|----------------------------------------------------------------------------------------------------|-----------------------------------------------------------------------------------|
|  EDA              | Distributions, corrélations, valeurs manquantes                                   |
|  Modèles          | Tableau comparatif, ROC, matrices de confusion                                    |
|  Prédiction       | Sliders capteurs → <span style="color: red;">●</span> <b>RISQUE ÉLEVÉ (78.3%)</b> |
|  Interprétabilité | Feature Importance + SHAP                                                         |

```
streamlit run dashboard/app.py → http://localhost:8501
```

## Bilan

- ✓ 4 modèles comparés · XGBoost retenu · F1 = **0.898** · Recall = **0.955** · AUC = **0.996**
- ✓ Cross-validation : **0.9026 ± 0.0099** — stable, pas d'overfitting
- ✓ Pipeline anti-leakage · SHAP · Dashboard opérationnel

XGBoost détecte 19 pannes sur 20 avant leur survenue

# Merci — Questions ?

Démonstration disponible

```
streamlit run dashboard/app.py
```

*Projet M2 Data Science — EFREI 2025-26*

*RNCP36739 — Bloc 4*